IRWA Project — Part 2: Indexing and Evaluation

Group: Xavier Gutiérrez (u233714), Pau Martínez (u232318), Mikel Areta (u232868)

- **GitHub repository:** github.com/XaviGuti25/irwa-search-engine-2026
- **Tag:** IRWA-2026-part-2
- **Code:** `project_progress/part_2/part2.ipynb` (our labels in `project_progress/part_2/my_labels.csv`)

We built a search engine over the 28,080 products of the dataset: an inverted index, AND queries and a TF-IDF ranking. It scores **MAP@10 = 0.785** on the two validation queries and **MAP@10 = 0.780** on our own five queries.

Documents and queries are cleaned with the same `preprocess` function from Part 1 (lowercase, no punctuation, stop words or numbers, *t-shirt* → *tshirt*, Porter stemming). Each product is searched through one text: title + description + useful product_details + brand + sub_category.

Part 1: Indexing

1. Build inverted index

The inverted index tells us, for every word, which products contain it: `{term: [pid_1, pid_2, ...]}`. Our vocabulary has 5,058 terms; for example, *sweatshirt* appears in 1,417 products. We also store how many times each word appears in each product, because TF-IDF needs it.

Queries are conjunctive (AND): a product is returned only if it contains **all** the query words. We clean the query with `preprocess` and intersect the product lists of its words.

Validation query	Products returned
women full sleeve sweatshirt cotton	467
men slim jeans blue	216

2. Propose test queries

We looked at the most popular words (in how many products they appear): *cotton* 19,610, *sleeve* 17,909, *men* 14,141, *women* 13,699, *regular* 13,089, *shirt* 4,809. Then we mixed popular words (gender, fit, colour) with more specific ones, so the queries look like real searches and return few enough results to judge them.

Query	Results	Why
women hooded jacket	80	A clothing type + an attribute (*hooded*)
men running shoes	76	Footwear, a much smaller category than topwear
women yellow kurta	39	Colour + an Indian garment: very specific

Query	Results	Why
men regular fit formal shirt	218	Long query of very common words: tests IDF
men sports shorts	196	*sports* appears in many other products: tests ranking

3. Rank your results

Each word gets a weight that is high when it appears often in the product (**tf**) and is rare in the collection (**idf**), with N = 28,080 products:

- **Weight:** $w(t,d) = (1 + \log tf(t,d)) \times \log(N / df(t))$
- **Score:** cosine similarity between the query and the product, $score(q,d) = \Sigma\, w(t,q) \times w(t,d) / (\|q\| \times \|d\|)$

The cosine stops long products from winning just for being long. Ties are broken by `pid`, so the ranking is always the same.

Top 3 results for three of our queries:

Query	Rank	Score	Title
women hooded jacket	1	0.6129	Full Sleeve Solid Women Sports Jacket
women hooded jacket	2	0.5916	Full Sleeve Solid Women Casual Jacket
women hooded jacket	3	0.5751	Full Sleeve Solid Women Casual Jacket
men running shoes	1	0.6615	Running Shoes For Women (White)
men running shoes	2	0.6493	Running Shoes For Men (Multicolor)
men running shoes	3	0.6493	Running Shoes For Men (Multicolor)
women yellow kurta	1	0.5803	Women Solid Cotton Blend Straight Kurta (Yellow)
women yellow kurta	2	0.5367	Women Striped Cotton Blend Straight Kurta (Yellow)
women yellow kurta	3	0.5367	Women Striped Cotton Blend Straight Kurta (Yellow)

Part 2: Evaluation

1. Implement the following evaluation metrics

Every metric uses the 1/0 labels of the results in ranking order (rel(i) at position i) and R, the number of relevant products for the query. We use K = 10.

Metric	Formula
i. Precision@K	relevant in top K / K

Metric	Formula
ii. Recall@K	relevant in top K / R
iii. Average Precision@K	average of P@i at each position i ≤ K that holds a relevant product, divided by min(R, K)
iv. F1-Score@K	2 × P@K × R@K / (P@K + R@K)
v. MAP	mean of AP@K over all queries
vi. MRR	mean of 1 / (position of the first relevant result)
vii. NDCG@K	DCG@K / ideal DCG@K, with DCG@K = Σ rel(i) / log2(i + 1)

We checked them on a small example, [1, 0, 1, 0, 0, 1] with R = 3: P@3 = 0.667, AP@6 = 0.722, NDCG@6 = 0.871, the same as by hand.

2. Apply the evaluation metrics to validation_labels.csv

Query	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
a. Query 1: women full sleeve sweatshirt cotton	1.000	0.769	1.000	0.870	1.000	1.000
b. Query 2: men slim jeans blue	0.600	0.600	0.569	0.600	1.000	0.716

MAP@10	MRR@10
0.785	1.000

3. You will act as expert judges by establishing the ground truth for each document and query

a. Assign a binary relevance label to each document for our test queries

We judged the **top 20 results** of each query (100 products) by reading the title, description and product details. A product is relevant (1) only if it is the right **type**, for the right **gender**, with the asked **attribute** (hooded, yellow, formal…). We cannot judge all 28,080 products, so R is the number of 1s among the 20 judged. The labels are in `my_labels.csv`.

Query	Relevant / 20	Typical 0
women hooded jacket	14	Hooded sweatshirts, jackets without hood
men running shoes	11	Women's shoes, socks, cricket shoes
women yellow kurta	13	Kurta + pyjama sets
men regular fit formal shirt	20	—

Query	Relevant / 20	Typical 0
men sports shorts	20	—

Query	P@10	R@10	AP@10	F1@10	RR@10	NDCG@10
women hooded jacket	0.700	0.500	0.700	0.583	1.000	0.801
men running shoes	0.800	0.727	0.598	0.762	0.500	0.710
women yellow kurta	0.600	0.462	0.600	0.522	1.000	0.727
men regular fit formal shirt	1.000	0.500	1.000	0.667	1.000	1.000
men sports shorts	1.000	0.500	1.000	0.667	1.000	1.000

MAP@10	MRR@10
0.780	0.900

b. Comment on each of the evaluation metrics, stating how they differ, and which information gives each of them. Analyze your results.

Metric	What it tells us	What it doesn't see
P@K	How many of the first K results are good	The order inside the top K and the relevant products we missed
R@K	How many of all the relevant products we found	The bad results; it can only go up with K, while P@K usually goes down
AP@K	Like precision, but rewards having the good results **at the top**	— (it cares about order)
F1@K	A balance of P@K and R@K: only high if both are high	The order
MAP	One number for the ranking quality of the whole system	Differences between queries: an easy query can hide a hard one
MRR	How soon the **first** good result appears	Everything after that first good result
NDCG@K	How close the ranking is to the perfect order (1 = perfect); lower positions count less	— (it also works with graded labels like 0/1/2)

What our results show

- **Query 1** is perfect: the first 10 results are all relevant (P@10 = AP@10 = NDCG@10 = 1). Recall is 0.769 only because there are 13 relevant products and 10 places.

- **Query 2** is much worse (AP@10 = 0.569). Our AND search returns only 8 of the 20 labelled products: the file marks some **women's** jeans as relevant, but they don't contain *men*, so we can never find them. MRR is still 1, which shows that MRR can hide problems further down.
- **men regular fit formal shirt** and **men sports shorts** are easy (everything = 1 except recall): the query words are exactly the words of the titles.
- **men running shoes** has a good P@10 (0.8) but the worst RR (0.5) and AP (0.598): the **first** result is a women's shoe whose sub_category is *"Men's Footwear"*. P@10 doesn't notice this; AP, RR and NDCG do.
- **women hooded jacket** (P@10 = 0.7) brings hooded sweatshirts at positions 8–11, because their description literally says *"Full Sleeve Hooded Jacket"*.
- **women yellow kurta** (P@10 = 0.6) mixes single kurtas with kurta + pyjama sets, which use the same words.
- Recall is ≤ 0.5 for most of our queries just because they have 11–20 relevant products and only 10 fit in the top 10, so P and R must be read together (F1).
- **Overall:** MAP is similar in both sets (0.785 and 0.780) and MRR is high (1.0 and 0.9). The system usually puts a good product first, but the rest of the ranking changes a lot from one query to another.

c. Analyze the current search system and identify its main problems or limitations. For each issue you find, propose possible ways to resolve it.

The two biggest problems are that AND is too strict and that the system doesn't understand gender or product type.

Aspect	Problem	Example	How to fix it
Query formulation	AND is too strict: one missing word and the product is lost	Query 2 only returns 8 of its 20 labelled products	Rank with OR, or use AND and fall back to OR when there are few results
Field types	All fields count the same	A word in a long description weighs as much as one in the title	Give the title more weight than the description (e.g. BM25F)
Retrieval accuracy	Gender is treated as any other word	A women's shoe is 1st for *men running shoes*	A gender field from *Ideal For* and the title, used as a filter
Retrieval accuracy	The system doesn't know the product type	Sweatshirts for *hooded jacket*, kurta + pyjama sets for *kurta*	Boost the product type (*Generic Name*, last word of the title)
Ranking quality	Simple TF-IDF and many ties (78.8% of titles are repeated)	Two shoes with exactly 0.6493 in *men running shoes*	BM25; break ties with rating, discount and stock

Aspect	Problem	Example	How to fix it
Query formulation	No synonyms or typo handling	*hoodie* vs *hooded*, *trainers* vs *shoes*	Synonym expansion, spelling correction, word embeddings
Indexing strategy	Words are indexed one by one	*full sleeve* is treated as two separate words	Index bigrams or word positions (phrase search)
Field types	Numeric fields are not used	Out-of-stock products rank like the rest	Use price, rating and stock as filters and ranking signals
Evaluation	Small ground truth judged by us	Only 20 judged products per query	More queries, more judges, graded labels (0/1/2)

Decisions and assumptions

- **One searchable text per product** (title + description + useful product_details + brand + sub_category), as decided in Part 1. Fields are not weighted yet; that is left for Part 3.
- **Validation evaluation:** we only know the labels of the 20 products per query in `validation_labels.csv`. We run our search engine and keep, in ranking order, the results that have a label. Relevant labelled products that our AND search does not return count as missed, so they lower recall.
- **Our own ground truth:** we judged only the top 20 results of each query, so R is the number of relevant products among those 20, not in the whole collection.
- **K = 10** for every metric, the size of a typical results page.

Use of AI tools

We used Claude to help write and debug the code, propose initial relevance labels, and draft this report.